

Integración de Sistemas

*Semana 5. SOA práctica, ESB y
mensajería empresarial (Reto 1)*

Sistemas de Información — PUCE

Docente: Damián Nicolalde

Semana 5. SOA práctica, ESB y mensajería empresarial (Reto 1)

Objetivo: Comprender e implementar los componentes prácticos de una arquitectura SOA (servicios, contratos, WSDL), el rol de un Enterprise Service Bus (ESB) en la orquestación y transformación de mensajes, los patrones fundamentales de mensajería empresarial (colas y tópicos), y una introducción operativa a RabbitMQ y Kafka, como base experimental para el Reto 1 del curso.

Resultado de aprendizaje asociado: Diseñar aplicaciones web basadas en el enfoque API-first, integrando servicios SOA, APIs REST y arquitecturas MVC, garantizando la interoperabilidad inicial del ecosistema.

Introducción

Después de cuatro semanas construyendo APIs REST, gRPC y GraphQL, esta semana regresa — con las manos en el código— a SOA: el estilo que dio origen a buena parte del vocabulario de la integración moderna (contrato, servicio, bus, orquestación). La diferencia frente a la Semana 1 es que ahora se construye un servicio SOA real (con el paquete soap sobre Node.js), se piensa en términos de ESB (aunque sea de forma simplificada), y se conecta ese mundo síncrono con el mundo asíncrono de la mensajería empresarial (RabbitMQ/Kafka). Esta semana cierra con el Reto 1: integrar un servicio SOA, un patrón tipo ESB y un evento de mensajería en un solo flujo, expuesto a través de un frontend MVC construido con NestJS.

5.1. Componentes SOA: servicios, contratos y WSDL

Una arquitectura SOA (Service-Oriented Architecture) se construye sobre tres piezas que deben coexistir de forma disciplinada: el servicio, el contrato y el registro/descubrimiento. Repasemos cada una desde una óptica práctica, orientada a lo que vas a construir esta semana.

Contexto: por qué SOA sigue vivo en 2026

A finales de los años 90 y comienzos de los 2000, las grandes organizaciones (bancos, aseguradoras, telecomunicaciones, gobierno) enfrentaban un problema muy concreto: tenían decenas de sistemas construidos en distintas épocas, con distintos lenguajes (COBOL, Java, .NET, PL/SQL), que necesitaban comunicarse entre sí para que el negocio funcionara como un todo. SOA surgió como la respuesta arquitectónica a ese problema: en vez de que cada sistema hablara con cada otro sistema de forma ad-hoc (la temida "malla de espagueti" punto a punto que ya viste en la Semana 1), cada capacidad de negocio se expone como un servicio con un contrato formal, y un bus (ESB) centraliza la mediación. El consorcio W3C estandarizó el conjunto de especificaciones conocido como "WS-*" (WS-Security, WS-ReliableMessaging, WS-Transaction, WS-Addressing, entre decenas de otras) precisamente para que estos servicios pudieran interoperar con garantías de seguridad, confiabilidad y transaccionalidad de nivel empresarial — algo que REST, en su simplicidad, no intentó resolver de la misma manera.

Casi 25 años después, gran parte de esa infraestructura SOA sigue en producción. No porque las organizaciones sean anticuadas, sino porque migrar un núcleo bancario, un sistema de facturación eléctrica o un motor de reservas aéreas es una operación de altísimo riesgo: cualquier error puede significar pérdidas millonarias o interrupciones regulatorias. Por eso, como profesional de integración, es más probable que tu primer trabajo real involucre conectar un sistema SOA existente (quizás escrito hace 15 años) con arquitecturas modernas, que "empezar de cero" con microservicios. Esta semana te da exactamente esa habilidad: saber construir y consumir un servicio SOA, y saber tenderle un puente hacia el mundo moderno (mensajería, eventos, frontend web) sin tener que reescribir el núcleo.

El servicio

Un servicio SOA expone una operación de negocio (no una tabla ni un CRUD genérico) a través de un contrato formal. Por ejemplo, ConsultarProducto, RegistrarPedido, ValidarInventario. El servicio encapsula su lógica interna: quien lo consume no sabe (ni le importa) si por dentro usa SQL Server, una API externa o un archivo plano.

Un principio central de SOA, heredado directamente de las buenas prácticas de ingeniería de software, es la granularidad correcta del servicio. Un servicio demasiado fino (por ejemplo, uno que solo devuelve el nombre de un producto y otro que solo devuelve su precio) obliga al consumidor a hacer múltiples llamadas encadenadas para completar una operación de negocio simple — el mismo problema de chatty interfaces que estudiaste al hablar de REST en la Semana 1. Un servicio demasiado grueso (uno que hace absolutamente todo lo relacionado con "productos, pedidos y pagos" en una sola operación gigante) se vuelve difícil de mantener, probar y reutilizar. La heurística práctica es diseñar el servicio alrededor de una capacidad de

negocio coherente: "gestión de catálogo de productos" es una buena unidad; "hacer todo lo que la tienda necesita" no lo es.

El registro y descubrimiento (UDDI)

La tercera pieza de una arquitectura SOA "de manual" es el registro de servicios, tradicionalmente implementado con UDDI (Universal Description, Discovery and Integration). La idea es que un proveedor publica su servicio (y su WSDL) en un directorio central, y los consumidores potenciales pueden descubrir qué servicios existen, sin que alguien tenga que comunicárselo manualmente por correo o documentación externa.

En la práctica actual, UDDI como protocolo formal cayó en desuso — la mayoría de las organizaciones nunca llegaron a desplegarlo en su forma original — pero la idea de fondo sigue absolutamente vigente bajo otros nombres: hoy se le llama "catálogo de APIs" o "developer portal" (Apigee, Kong Developer Portal, Azure API Management), y cumple exactamente la misma función: centralizar el descubrimiento de qué servicios/APIs existen, quién es su dueño, y cómo consumirlos. Cuando en la Semana 1 hablamos de "gobernanza" y "catálogo corporativo" como principio rector de una integración profesional, esa idea nace directamente de la disciplina SOA de los 2000.

El contrato (WSDL)

El WSDL (Web Services Description Language) es el documento XML que describe exhaustivamente un servicio SOAP: qué operaciones existen, qué mensajes de entrada/salida esperan, y sobre qué protocolo de transporte viajan (normalmente HTTP + SOAP/XML).

Un WSDL completo se compone de cinco elementos que conviene poder identificar a simple vista, aunque hoy los generemos automáticamente:

Elemento	Qué describe
types	Los tipos de datos que usan los mensajes (normalmente definidos con XML Schema/XSD).
message	La "forma" de una solicitud o respuesta (qué partes/campos la componen).
portType (interface en WSDL 2.0)	El conjunto de operaciones disponibles y qué mensajes usa cada una.
binding	Cómo se serializan esos mensajes físicamente (SOAP sobre HTTP).
service	La URL concreta donde el servicio está desplegado y accesible.

```
<definitions name="ServicioProductos"
targetNamespace="http://empresa.com/wsdl/productos">
  <message name="ObtenerProductosRequest" />
  <message name="ObtenerProductosResponse">
    <part name="productos" type="tns:ArrayOfProducto"/>
  </message>
</definitions>
```

```

</message>
<portType name="ProductosPortType">
  <operation name="ObtenerProductos">
    <input message="tns:ObtenerProductosRequest"/>
    <output message="tns:ObtenerProductosResponse"/>
  </operation>
</portType>
<binding name="ProductosBinding" type="tns:ProductosPortType">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http"/>
</binding>
</definitions>

```

Lo importante no es memorizar la sintaxis XML (nadie la escribe a mano hoy), sino entender que el WSDL es generado automáticamente por el framework a partir de tu código, de la misma forma en que Swagger generaba el contrato OpenAPI a partir de tu controlador REST en la Semana 1, o protoc generaba el stub a partir del .proto en la Semana 3. El patrón se repite: código → contrato autogenerado → cliente que lo consume.

Anatomía de un mensaje SOAP

Cada solicitud y respuesta que viaja hacia/desde un servicio SOA sigue la estructura de un sobre SOAP (SOAP Envelope), compuesto por dos partes:

- Header (opcional): metadatos de la solicitud que no son parte del "negocio" — tokens de seguridad (WS-Security), identificadores de correlación, o información de enrutamiento. Equivalente conceptual a las cabeceras HTTP en REST.
- Body (obligatorio): el contenido real de la operación — los parámetros de entrada, o los datos de la respuesta.

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <!-- metadatos: seguridad, correlacion, etc. (opcional) -->
  </soap:Header>
  <soap:Body>
    <ObtenerProductos xmlns="http://tempuri.org/" />
  </soap:Body>
</soap:Envelope>

```

Cuando construyas el frontend del Reto 1 y armes este sobre a mano (o inspecciones el que arma la librería soap), estarás viendo exactamente esta estructura — es una experiencia deliberadamente "manual" para que entiendas qué es lo que las herramientas normalmente ocultan.

Manejo de errores: SOAP Faults

Así como REST tiene códigos de estado HTTP (Semana 2) y GraphQL tiene su propio bloque errors en la respuesta (Semana 4), SOAP tiene su propio mecanismo de error: el SOAP Fault. Un Fault es una respuesta especial dentro del Body que indica que la operación no pudo completarse, con tres campos clave: faultcode (una categoría del error, ej. Client o Server),

faultstring (un mensaje legible) y opcionalmente detail (información adicional específica del dominio).

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>El producto solicitado no existe</faultstring>
</soap:Fault>
```

A diferencia de REST, donde un 404 viaja en la cabecera HTTP sin necesidad de "abrir" el cuerpo de la respuesta, en SOAP siempre se recibe un HTTP 200 a nivel de transporte (salvo errores de infraestructura), y es responsabilidad del cliente inspeccionar el Body para saber si viene un Fault o una respuesta exitosa. Esta es una de las razones por las que herramientas de monitoreo genéricas basadas solo en códigos HTTP no son suficientes para observar servicios SOAP correctamente — otra pista de por qué la observabilidad "desde el día 1" (principio rector de la Semana 1) importa tanto.

SOAP es agnóstico del lenguaje: dos implementaciones del mismo contrato

Una de las lecciones más importantes de esta semana es que SOAP no le pertenece a ningún lenguaje en particular: es un protocolo basado en XML sobre HTTP, así que cualquier plataforma con soporte HTTP puede implementarlo. Dos ejemplos de librerías que resuelven exactamente el mismo problema en ecosistemas distintos:

SoapCore (.NET): ASP.NET Core no trae soporte SOAP de fábrica (a diferencia del antiguo .NET Framework, que sí tenía WCF nativo). SoapCore expone servicios SOAP sobre ASP.NET Core moderno, generando el WSDL automáticamente a partir de una interfaz decorada con atributos:

```
[ServiceContract]
public interface IProductoService
{
    [OperationContract]
    List<ProductoDto> ObtenerProductos();
}
// Program.cs
builder.Services.AddSoapCore();
app.UseSoapEndpoint<IProductoService>("/ProductoService.asmx", new
SoapEncoderOptions());
```

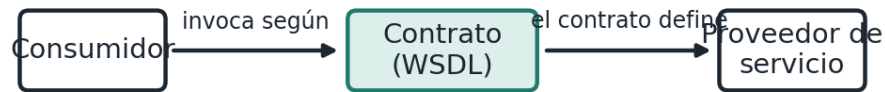
soap (Node.js): el paquete que usaremos en el Reto 1 de esta semana. Expone un servicio SOAP a partir de una definición WSDL y un objeto JavaScript con las funciones que implementan cada operación:

```
const service = {
  ProductoService: {
    ProductoServicePort: {
      ObtenerProductos: function () {
        return { productos: PRODUCTOS };
      },
    },
  },
},
```

```
};  
soap.listen(app, '/wsdl', service, wsdlXml);
```

El resultado observable —un endpoint que responde a solicitudes SOAP y publica un WSDL en una URL conocida— es idéntico desde el punto de vista de cualquier cliente, sin importar si detrás corre C#, Java o Node.js. Esto es precisamente lo que hace valiosa a la interoperabilidad basada en contratos: el consumidor nunca necesita saber qué hay del otro lado.

Diagrama 1. Componentes de un servicio SOA: el Consumidor llama al Proveedor de servicio a través del Contrato (WSDL), que describe operaciones, mensajes y bindings — sin importar en qué lenguaje esté implementado cada lado.



Buenas prácticas al diseñar un servicio SOA

- Nombrar operaciones con verbos de negocio (ConsultarSaldo, no GetData).
- Mantener el contrato estable: una vez publicado un WSDL, cambiarlo rompe a todos los consumidores existentes (a diferencia de REST/GraphQL, SOAP no tiene un mecanismo estándar de evolución tolerante).
- Separar el contrato de la implementación, igual que separarías una interfaz de su clase concreta.
- Documentar cada operación: en SOA clásico, el WSDL es frecuentemente la única documentación disponible para otros equipos.
- Diseñar pensando en la granularidad correcta: ni un servicio "todólogo", ni una colección de micro-operaciones que obliguen a encadenar llamadas.
- Versionar el namespace del servicio cuando un cambio sea incompatible (ej. `.../productos/v2`), en vez de mutar el contrato existente en el mismo namespace.

Síntesis: los contratos a través de las cinco semanas

Ya llevas cinco semanas viendo la misma idea repetirse bajo distintos disfraces: un contrato explícito es lo que permite que dos sistemas construidos por equipos distintos, en momentos distintos, se entiendan sin llamarse por teléfono para "ponerse de acuerdo". Vale la pena mirar la tabla comparativa completa antes de seguir:

Tabla 0. Contratos de integración estudiados hasta ahora

Semana	Estilo	Formato del contrato	Generado desde	Tipado
1-2	REST	OpenAPI (YAML/JSON)	Código del controlador (Swagger)	Dinámico, reforzado por esquema
3	gRPC	Protobuf (.proto)	Se escribe el .proto, se genera el código	Fuerte y binario
4	GraphQL	SDL	Se define el	Fuerte

			esquema (code-first o schema-first)	
5	SOA	WSDL (XML)	Código del servicio	Fuerte, basado en XSD

Ninguno de estos enfoques es "el correcto": cada uno resuelve una tensión distinta entre legibilidad humana, rendimiento, tooling disponible y madurez del ecosistema. Lo que sí es constante es el patrón subyacente: contrato explícito → generación automática de código → validación en tiempo de desarrollo, no en producción.

Ejercicio resuelto: leyendo un WSDL real de principio a fin

Antes de escribir una sola línea de servidor SOAP, conviene practicar la habilidad inversa: leer un WSDL ya generado y predecir qué hace el servicio, exactamente como harías con la documentación de Swagger de un servicio REST que nunca has visto.

Enunciado: te entregan el siguiente fragmento de WSDL de un servicio de terceros. Sin ver ni una línea de su código fuente, responde: (a) ¿cómo se llama la única operación expuesta?, (b) ¿qué datos hay que enviarle?, (c) ¿qué forma tiene la respuesta?

```
<definitions name="ServicioInventario"
targetNamespace="http://acme.com/wsdl/inventario">
  <message name="ConsultarStockRequest">
    <part name="codigoProducto" type="xsd:string"/>
  </message>
  <message name="ConsultarStockResponse">
    <part name="cantidadDisponible" type="xsd:int"/>
    <part name="bodega" type="xsd:string"/>
  </message>
  <portType name="InventarioPortType">
    <operation name="ConsultarStock">
      <input message="tns:ConsultarStockRequest"/>
      <output message="tns:ConsultarStockResponse"/>
    </operation>
  </portType>
</definitions>
```

Solución razonada:

1. Operación: buscamos portType/operation: se llama ConsultarStock. Es una única operación, coherente con el principio de granularidad correcta.
2. Entrada: la operación referencia ConsultarStockRequest como input; ese message tiene una sola part, codigoProducto de tipo string. Hay que enviar el código del producto como único parámetro (equivalente a GET /productos/{codigo}/stock en REST).
3. Salida: el output referencia ConsultarStockResponse, con dos part: cantidadDisponible (entero) y bodega (texto). La respuesta trae cuánto stock hay y en qué bodega está.

Este mismo ejercicio de "leer el contrato antes de leer el código" es exactamente lo que harás al integrar cualquier servicio SOA de terceros en tu vida profesional — muchas veces el WSDL es lo único que el equipo proveedor te entrega.

5.2. ESB: conceptos, orquestación y transformación

Un Enterprise Service Bus (ESB) es la pieza de infraestructura que media entre múltiples servicios: enruta mensajes, transforma formatos, aplica políticas de seguridad y, opcionalmente, orquesta secuencias de llamadas. Ya lo viste conceptualmente en la Semana 1 (topologías) y en la Semana 2 (patrones SOA-REST); esta semana lo hacemos operativo.

Las tres funciones centrales de un ESB

1. Enrutamiento (routing): decide a qué servicio debe llegar un mensaje, según su contenido, su origen o reglas de negocio.
2. Transformación (transformation): convierte un mensaje de un formato/esquema a otro — por ejemplo, de XML/SOAP a JSON, o de un esquema "Cliente v1" a "Cliente v2".
3. Orquestación (orchestration): coordina una secuencia de llamadas a distintos servicios para completar un proceso de negocio.

```
Solicitud entrante (XML/SOAP)
  |
  v
[ ESB: enrutar segun operacion ]
  |
  v
[ ESB: transformar SOAP -> JSON interno ]
  |
  v
[ Invocar servicio interno / publicar evento ]
```

Diagrama 2. Un ESB simplificado: las aplicaciones A, B y C no se conectan directamente entre sí; todas hablan con el ESB, que enruta y transforma según corresponda.



ESB "real" vs ESB "de bolsillo" para este curso

En la industria, un ESB es un producto robusto con años de desarrollo detrás. Vale la pena conocer el panorama, aunque no instalemos ninguno en esta práctica:

Producto	Origen	Fortaleza típica
MuleSoft Anypoint Platform	Salesforce (2018)	Ecosistema de conectores prediseñados y diseño visual de flujos.
WSO2 Enterprise Integrator	Open source	Fuerte en gobernanza de APIs y mediación SOA-REST.
IBM App Connect / Integration Bus	IBM	Robustez transaccional para banca/seguros, integración con mainframes.
Azure Service Bus + Logic Apps	Microsoft	Mensajería como servicio (PaaS) + orquestación visual en la nube.
Apache Camel	Open source (Apache)	Enterprise Integration Patterns embebibles en apps Java/Spring.

Para el alcance de esta asignatura, no vamos a instalar ninguno de estos productos: vamos a implementar el patrón ESB de forma simplificada dentro de nuestro propio código — una capa intermedia que recibe la llamada SOAP, decide qué hacer (enrutar/transformar) y publica un evento. Esto es exactamente lo que vas a construir en el Reto 1. Entender el patrón en pequeño te permite, más adelante en tu carrera, entender rápidamente cualquiera de estas herramientas — todas resuelven el mismo problema con distinto nivel de sofisticación y distinto precio de licencia.

Patrones de integración empresarial (Hohpe & Woolf)

El libro Enterprise Integration Patterns (Hohpe & Woolf, 2003) catalogó decenas de patrones que cualquier ESB —comercial o "de bolsillo"— termina implementando en alguna forma. Los más relevantes para lo que construirás esta semana:

- Message Translator: transforma la estructura o el formato de un mensaje sin cambiar su significado (ej. de XML/SOAP a un objeto JSON interno). Es lo que hace tu capa de transformación en el Reto 1.
- Content-Based Router: examina el contenido del mensaje para decidir a qué destino enviarlo.
- Message Filter: descarta mensajes que no cumplen cierto criterio antes de que lleguen al siguiente paso del flujo.
- Aggregator: combina varios mensajes relacionados en uno solo antes de continuar (lo inverso de un Splitter).
- Splitter: divide un mensaje compuesto (ej. un pedido con 5 líneas) en varios mensajes individuales.
- Canonical Data Model: define un formato de datos "neutral" al que todos los sistemas traducen, evitando que N sistemas necesiten N x (N-1) traductores.

Fíjate que estos patrones no son exclusivos de un ESB "grande": los usarás en cualquier lugar donde un componente reciba datos de un formato y deba entregarlos en otro — incluyendo tu propia capa EventPublisher del Reto 1, que actúa como un Message Translator simplificado.

Riesgos del ESB (repaso aplicado)

- ESB-monolito: si toda la lógica de transformación termina viviendo en el bus, se vuelve un cuello de botella y un punto único de fallo.
- Acoplamiento oculto: un ESB mal gobernado esconde reglas de negocio que nadie más conoce ni versiona.
- Mitigación: mantener las transformaciones simples, versionadas y testeadas; usar el ESB para mediar, no para acumular lógica de dominio.

Estudio de caso: modernizando un núcleo bancario con un ESB

Un banco tiene un núcleo bancario (core banking) escrito en COBOL sobre mainframe, en producción desde hace más de 20 años, que expone únicamente una interfaz propietaria. El banco quiere lanzar una app móvil moderna (REST/JSON), seguir dando servicio a cajeros automáticos (protocolo ISO 8583) y alimentar un sistema de reportería regulatoria que espera XML con un esquema propio. Reescribir el núcleo no es una opción realista: es la parte más crítica del sistema.

La solución típica es introducir un ESB como capa de mediación: (1) el ESB expone un servicio SOA moderno hacia el núcleo, ocultando el protocolo propietario detrás de operaciones de negocio claras; (2) la app móvil consume una API REST que el ESB traduce internamente (Message Translator) hacia ese servicio SOA; (3) los cajeros siguen hablando ISO 8583

directamente con el ESB, que también lo traduce; (4) ante cada transacción relevante, el ESB publica un evento a un bus de mensajería para que el sistema regulatorio genere su XML de forma asíncrona, sin bloquear la transacción del cliente.

Este caso ilustra por qué SOA y los ESB siguen siendo la solución más pragmática para modernizar sistemas críticos sin apagarlos ni reescribirlos — y por qué el Reto 1, a pequeña escala, reproduce esta misma arquitectura de tres capas (servicio SOA + mediación + mensajería).

5.3. Patrones de mensajería empresarial (colas y tópicos)

La mensajería empresarial es el mecanismo que permite que los servicios se comuniquen de forma asíncrona y desacoplada. Dos patrones estructurales dominan este espacio:

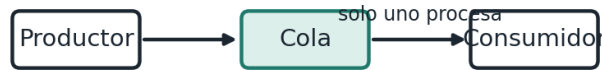
Patrón 1 - Cola (Point-to-Point)

Un productor envía un mensaje a una cola; un único consumidor lo procesa y el mensaje desaparece de la cola. Si hay varios consumidores compitiendo por la misma cola, cada mensaje lo procesa solo uno de ellos (patrón Competing Consumers), útil para repartir carga de trabajo.

Casos de uso: procesar pedidos uno a la vez, tareas en segundo plano, trabajos que no deben duplicarse.

Diagrama 3a. Patrón Cola: Productor -> Cola -> Consumidor (uno de varios, si compiten por la cola).

Patrón Cola (Point-to-Point)

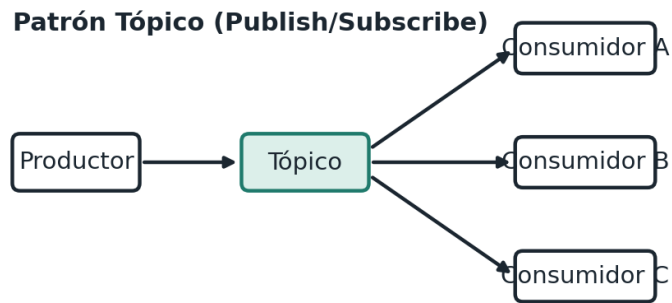


Patrón 2 - Tópico (Publish/Subscribe)

Un productor publica un mensaje en un tópico; todos los consumidores suscritos reciben una copia del mismo mensaje. Es el patrón detrás de las subscriptions de GraphQL (Semana 4) y de los eventos de dominio en arquitecturas orientadas a eventos (Semana 6).

Casos de uso: notificar a múltiples sistemas que "un producto cambió de precio", sin que el productor sepa (ni le importe) cuántos ni quiénes escuchan.

Diagrama 3b. Patrón Tópico: Productor -> Tópico -> Consumidor A + Consumidor B + Consumidor C
(todos reciben el mismo mensaje).



Otros patrones de mensajería relevantes

Patrón	Qué resuelve
Dead Letter Queue (DLQ)	Mensajes que fallan repetidamente se apartan a una cola especial para análisis.
Message Router	Un componente decide a qué cola/tópico enviar un mensaje según su contenido.
Retry con backoff	Reintentar el procesamiento de un mensaje fallido con espera creciente.
Idempotent Consumer	El consumidor procesa un mensaje duplicado sin producir efectos duplicados.

Tabla 1. Cola vs. Tópico

Característica	Cola (P2P)	Tópico (Pub/Sub)
Consumidores	Uno procesa cada mensaje	Todos los suscritos reciben copia
Propósito típico	Repartir trabajo	Difundir un evento
Acoplamiento	El productor no sabe quién procesa	El productor no sabe ni cuántos ni quiénes escuchan
Ejemplo	Cola de impresión, pagos	PedidoCreado a Inventario + Facturación + Analítica

Garantías de entrega: at-most-once, at-least-once, exactly-once

Un aspecto que casi todo estudiante subestima la primera vez que trabaja con mensajería es que la red no es confiable: un mensaje puede perderse, un consumidor puede caerse a mitad de procesar, o una confirmación (ack) puede no llegar de vuelta al broker aunque el mensaje sí se haya procesado. Los sistemas de mensajería ofrecen tres niveles de garantía, y elegir el correcto es una decisión de diseño, no un detalle técnico menor:

- At-most-once: el mensaje se entrega 0 o 1 veces. Si algo falla, se pierde y no se reintenta. Aceptable solo cuando perder un mensaje ocasional no es grave.
- At-least-once: el broker garantiza que el mensaje llega, reintentando si no recibe confirmación — pero puede llegar duplicado. Es el nivel que ofrecen RabbitMQ y Kafka por defecto.
- Exactly-once: el mensaje se procesa una única vez, sin duplicados ni pérdidas. La garantía más deseable, pero también la más costosa de implementar correctamente.

La consecuencia práctica de esto es el patrón Idempotent Consumer: dado que "at-least-once" es lo más común, el consumidor debe estar preparado para recibir el mismo evento dos veces y producir el mismo resultado final (por ejemplo, verificando si ya existe un registro con ese id de evento antes de insertarlo de nuevo). Diseñar consumidores idempotentes no es opcional en sistemas de mensajería reales: es una obligación.

Otros patrones que verás en proyectos reales

- Claim Check: en vez de enviar un payload muy pesado directamente en el mensaje, se guarda el contenido en un almacenamiento externo y se envía solo una referencia ("ticket de reclamo") en el mensaje.
- Message Expiration (TTL): un mensaje que no fue consumido dentro de un tiempo determinado se descarta automáticamente o se mueve a una DLQ.
- Correlation Identifier: cada mensaje lleva un identificador que permite relacionar una solicitud con su respuesta asíncrona correspondiente, o rastrear todos los mensajes de un mismo proceso de negocio.

Ejercicio resuelto: diseñando el evento ProductoConsultado

Enunciado: diseña el payload JSON del evento ProductoConsultado, que se publicará cada vez que el servicio SOA reciba una llamada a ObtenerProductos. El evento será consumido por un worker que solo necesita registrar un log de auditoría.

Solución razonada:

1. Nombre del evento en tiempo pasado: ProductoConsultado (no ConsultarProducto). Un evento describe algo que ya ocurrió; un comando describe algo que se pide que ocurra.
2. Campos mínimos necesarios para el propósito declarado (auditoría): no hace falta el catálogo completo, solo lo suficiente para saber qué y cuándo se consultó.
3. Un identificador de correlación, para rastrear la consulta en logs distribuidos si se agregan más consumidores.

```
{
  "eventoId": "6f2a1e40-9c3d-4b1a-8e2f-1a2b3c4d5e6f",
  "tipo": "ProductoConsultado",
  "fecha": "2026-09-08T14:32:10Z",
  "origen": "soa-productos-service",
```

```
"totalProductosDevueltos": 3  
}
```

Lo que deliberadamente NO se incluye: el arreglo completo de productos. Si otro consumidor lo necesitara, sería señal de crear un evento distinto y más específico (CatalogoConsultadoCompleto), en vez de sobrecargar ProductoConsultado — el mismo principio de granularidad correcta aplicado ahora al diseño de eventos.

5.4. Introducción a RabbitMQ y Kafka

RabbitMQ y Kafka son los dos brokers de mensajería más usados en la industria, pero parten de modelos mentales distintos.

RabbitMQ: el broker de colas clásico

RabbitMQ implementa el protocolo AMQP (Advanced Message Queuing Protocol). Su unidad central es el Exchange, que recibe mensajes del productor y los enruta a una o más colas según un tipo de enrutamiento.

Tipo	Comportamiento	Ejemplo de uso
direct	Entrega a la(s) cola(s) cuya routing key coincide exactamente.	pedido.pagado va solo a la cola suscrita exactamente a pedido.pagado.
fanout	Ignora la routing key: entrega a todas las colas conectadas.	Notificar "el catálogo cambió" a Caché, Buscador y Analítica a la vez.
topic	Entrega según un patrón con comodines (* = una palabra, # = cero o más).	pedido.*.urgente recibe pedido.pagado.urgente pero no pedido.pagado.normal.

En el Reto 1 usarás el exchange por defecto de RabbitMQ (tipo direct implícito, nombre vacío), que enruta directamente por el nombre de la cola — la forma más simple posible de publicar, apropiada para un primer contacto con la herramienta.

Diagrama 4a. Arquitectura de RabbitMQ: Productor -> Exchange -> Cola -> Consumidor.

RabbitMQ



Cómo se ve RabbitMQ "por dentro" durante una operación

Cuando se declara una cola y se publica un mensaje, ocurren varios pasos coordinados entre la aplicación y el broker: (1) la aplicación abre una conexión TCP persistente hacia RabbitMQ; (2) sobre esa conexión se abre un canal (un "carril" lógico más liviano que abrir una conexión nueva por operación); (3) se declara la cola (si no existe, se crea; si ya existe con la misma configuración, no pasa nada); (4) se publica el mensaje indicando el exchange y la routing key. Este mismo patrón —conexión persistente, canal ligero, operación idempotente de declaración— se repite en prácticamente cualquier cliente de mensajería que uses en tu carrera, incluyendo Kafka y Azure Service Bus.

Kafka: el log distribuido de eventos

Kafka no es "una cola": es un log de eventos distribuido y particionado. Los productores escriben en un tópico, que se divide en particiones; los consumidores se organizan en grupos de consumo, y cada partición es leída por un solo consumidor dentro de un grupo (pero varios grupos pueden leer el mismo tópico de forma independiente). A diferencia de RabbitMQ, los mensajes no se eliminan al ser leídos: permanecen en el log durante un periodo de retención configurable, lo que permite "rebobinar" y reprocesar eventos.

Diagrama 4b. Arquitectura de Kafka: Productor -> Tópico (particiones) -> Grupo de consumidores.

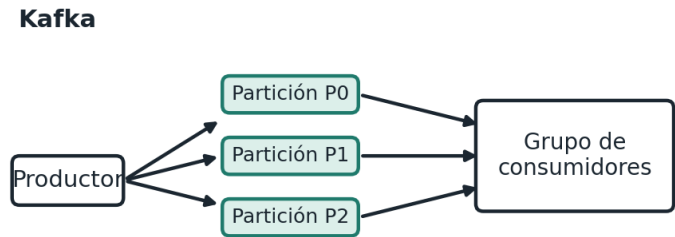


Tabla 2. RabbitMQ vs. Kafka

Característica	RabbitMQ	Kafka
Modelo	Cola / Exchange	Log distribuido de eventos
Retención	Se borra al confirmarse (ack)	Configurable, reprocesable
Orden garantizado	Por cola	Por partición
Throughput típico	Alto	Muy alto (streams masivos)
Enrutamiento	Flexible (direct/fanout/topic)	Por tópico + partición
Caso de uso típico	Tareas, colas de trabajo, RPC asíncrono	Streaming, event sourcing, analítica
Curva de aprendizaje	Más simple para empezar	Más compleja (particiones, offsets, grupos)

Piezas adicionales del ecosistema Kafka

- Brokers: cada nodo físico/lógico del clúster de Kafka. Un clúster de producción típicamente tiene múltiples brokers para repartir carga y tolerar fallos.
- Replicación: cada partición se copia en varios brokers; si el broker "líder" cae, otro con réplica sincronizada toma su lugar sin perder datos.
- Coordinación del clúster (ZooKeeper -> KRaft): Kafka dependía de Apache ZooKeeper para coordinar metadatos del clúster; las versiones recientes incorporaron el protocolo propio KRaft, eliminando esa dependencia.

Otro matiz importante: en Kafka, el productor puede elegir su nivel de confirmación (acks): acks=0 (no espera confirmación, máxima velocidad, riesgo de pérdida), acks=1 (espera solo al broker líder) o acks=all (espera a todas las réplicas sincronizadas, máxima durabilidad). Es la misma tensión entre rendimiento y garantía de entrega discutida en 5.3, solo que aquí es un parámetro explícito.

¿Cuál usar?

Ninguno reemplaza al otro. RabbitMQ brilla en integración de aplicaciones "clásica"; Kafka brilla cuando el volumen de eventos es masivo y necesitas reproducibilidad (auditoría, event sourcing, analítica). Para el Reto 1 de esta semana usaremos RabbitMQ, por su simplicidad de instalación y su ajuste perfecto al caso de uso (publicar un evento cuando se consulta un producto).

Y en la nube: los servicios de mensajería gestionados

RabbitMQ y Kafka se pueden auto-alojar (como haremos en el Reto 1, con Docker), pero los tres grandes proveedores de nube ofrecen versiones gestionadas de mensajería que resuelven el mismo problema sin que el equipo tenga que operar el broker:

Servicio	Proveedor	Modelo más parecido a
Amazon SQS	AWS	RabbitMQ (colas simples, gestionadas)
Amazon SNS	AWS	Tópico fanout (pub/sub gestionado)
Azure Service Bus	Microsoft	RabbitMQ (colas y tópicos, más features empresariales)
Google Cloud Pub/Sub	Google Cloud	Tópico fanout distribuido y altamente escalable
Amazon MSK / Confluent Cloud	AWS / Confluent	Kafka gestionado

La razón de mencionarlos, aunque el Reto 1 use RabbitMQ auto-alojado, es que los conceptos que aprendes esta semana (exchange, cola, tópico, ack, DLQ, idempotencia) se transfieren directamente a cualquiera de estos servicios gestionados: cambia el nombre del producto y el proveedor de infraestructura, pero no cambia el modelo mental.

5.5. Práctica del Reto 1: servicio SOA + ESB + evento en RabbitMQ + Frontend MVC

Esta sección resume el flujo integral que vas a construir en la guía de práctica, uniendo todo lo visto en 5.1-5.4. En esta edición del curso, el Reto 1 se implementa íntegramente en JavaScript/Node.js: el paquete soap para exponer y consumir SOAP, amqplib para hablar con RabbitMQ, y NestJS (modo MVC con hbs) para el frontend.

1. Un servicio SOA (soa-productos-service, Node + Express + soap) expone la operación ObtenerProductos sobre el dominio de Productos que vienes construyendo desde la Semana 1.
2. Una capa ESB simplificada (EventoPublisher) intercepta la llamada una vez resuelta, y publica un evento (ProductoConsultado) a RabbitMQ usando amqplib.
3. Un consumidor (consumidor-eventos, proceso Node independiente con amqplib) escucha la cola y reacciona al evento — por ejemplo, registrando un log de auditoría.
4. Un frontend MVC (frontend-mvc, NestJS en modo MVC con la vista hbs) consume el servicio SOAP y muestra los productos al usuario final, cerrando el ciclo: SOA (síncrono) -> ESB (mediación) -> Mensajería (asíncrono) -> MVC (presentación).

Diagrama 5. Frontend MVC (NestJS) -> llamada SOAP -> Servicio SOA/ESB (Node+soap) -> publica evento (amqplib) -> RabbitMQ -> Consumidor (worker Node) procesa el evento de forma independiente.



Este es el primer reto del curso: integrar cuatro piezas distintas -que además corren como procesos independientes- en un único flujo coherente. Es exactamente el tipo de trabajo que hace un ingeniero de integración en la vida real.

¿Qué se evalúa realmente en este reto?

La rúbrica del Reto 1 no premia la cantidad de código, sino la comprensión de la integración:

Criterio	Qué significa en la práctica
El contrato SOAP funciona	El WSDL se genera correctamente y se puede invocar la operación
El evento se publica correctamente	Se ve, en la consola de RabbitMQ, que el mensaje llegó a la cola esperada
El consumidor reacciona de forma independiente	Al detener el frontend/servicio, el consumidor sigue procesando lo acumulado
El frontend muestra datos reales	La página HTML no tiene datos quemados: viene de la respuesta SOAP real
Puedes explicar el flujo sin ver el código	Puedes responder qué pasa si el consumidor está apagado, usando 5.1-5.4

Errores comunes (y cómo evitarlos)

- Publicar el evento antes de confirmar la respuesta SOAP: si la operación falla después, queda un evento "fantasma". Publica el evento después de tener la respuesta lista.
- No declarar la cola antes de publicar: si el productor publica a una cola que el consumidor aún no ha declarado, el mensaje puede perderse. Ambos lados deben declarar la cola con los mismos parámetros.
- Confundir el rol del ESB con el de la cola: EventoPublisher decide qué evento publicar; RabbitMQ solo lo transporta.
- Frontend acoplado a los tres servicios a la vez: el frontend debe hablar solo con el servicio SOA, nunca conectarse directamente a RabbitMQ.

Cronograma sugerido para completar el Reto 1

El Reto 1 tiene más piezas móviles que cualquier práctica anterior del curso. Esta distribución de tiempo (para una sesión de laboratorio de 3 horas, más trabajo autónomo) ayuda a no quedarse atascado en una sola pieza:

Bloque	Tiempo aprox.	Qué deberías tener al terminar
Preparar entorno (Node, Docker, RabbitMQ)	20-30 min	RabbitMQ corriendo, consola accesible
Construir soa-productos-service (SOAP+WSDL)	40-50 min	El WSDL se genera y responde a una llamada de prueba
Agregar EventoPublisher (RabbitMQ)	30-40 min	El evento aparece en la cola tras cada consulta
Construir consumidor-eventos	20-30 min	La consola del consumidor muestra el evento recibido
Construir frontend-mvc (NestJS+hbs)	40-50 min	La página muestra los productos reales
Documentar y verificar el flujo completo	15-20 min	Puedes explicar el flujo de punta a punta sin ver el código

Si un bloque toma mucho más de lo estimado, es buena señal para pedir ayuda dirigida (docente o, de forma acotada, un agente de IA) en vez de seguir empujando solo.

5.6. Uso responsable de agentes de IA — Semana 5

Esta semana integra varias tecnologías a la vez (soap, ESB conceptual, amqplib/RabbitMQ, NestJS en modo MVC), lo que la hace tentadora para pedirle todo a la IA de una sola vez. El Reto 1 evalúa tu capacidad de integrar, no de copiar un tutorial.

Usos recomendados

- Interpretar errores de configuración del paquete soap o de conexión a RabbitMQ con amqplib.
- Explicar la diferencia entre exchange direct/fanout/topic con un ejemplo propio.
- Revisar (no escribir) el diseño del evento a publicar.
- Explicar la diferencia entre el modo MVC de NestJS (con hbs) y el modo API REST de NestJS.

Usos NO permitidos

- Pedir el Reto 1 completo sin levantar tú mismo RabbitMQ ni comprobar el mensaje.
- Copiar configuración de conexión sin adaptarla y entenderla.
- Aceptar sin verificar que "Kafka es mejor que RabbitMQ" sin contrastarlo con la Tabla 2.

Checklist antes de entregar algo influenciado por IA

1. ¿Puedo explicar, sin ver el código, qué hace cada una de las 4 piezas del Reto 1?
2. ¿Comprobé yo mismo, en la consola de RabbitMQ, que el mensaje efectivamente llegó?
3. ¿Declaré el uso de IA según el formato del curso?

Resumen de la semana

Sección	Idea central	Lo que construiste/leíste
5.1	Un servicio SOA expone una operación de negocio a través de un contrato formal (WSDL) - el mismo patrón "contrato explícito" de REST/gRPC/GraphQL, ahora en XML.	Leíste y razonaste un WSDL de principio a fin.
5.2	Un ESB media, transforma y opcionalmente orquesta entre servicios, evitando la malla punto a punto.	Analizaste un caso real de modernización bancaria con ESB.
5.3	La mensajería asíncrona (colas y tópicos) desacopla productores de consumidores, a costa de diseñar para duplicados (Idempotent Consumer).	Diseñaste el payload del evento ProductoConsultado.
5.4	RabbitMQ (broker de colas) y Kafka (log distribuido) resuelven mensajería con modelos mentales distintos; ambos tienen equivalentes gestionados en la nube.	Comparaste RabbitMQ vs. Kafka y sus equivalentes en AWS/Azure/GCP.
5.5	El Reto 1 integra las cuatro piezas (SOA + ESB + mensajería + MVC) como procesos Node independientes.	Planificaste tu propio cronograma de trabajo para el reto.
5.6	El Reto 1 evalúa integración, no copiado de tutoriales.	Revisaste el checklist de uso responsable de IA.

Esta tabla es, en sí misma, un buen punto de partida para repasar antes de tu evaluación: si puedes explicar cada celda de la columna central sin ver el resto del capítulo, dominas el contenido de la semana.

Preguntas frecuentes

¿Necesito instalar SoapCore o algo de .NET para el Reto 1?

No. El Reto 1 completo se implementa en Node.js/JavaScript: el servicio SOA usa el paquete soap, no SoapCore. SoapCore se menciona en 5.1 solo como ejemplo de que SOAP también existe en el ecosistema .NET.

¿Por qué SOAP sigue usando XML si JSON es más ligero?

Porque SOAP nació en una época donde XML era el estándar dominante, y trae consigo todo el ecosistema WS-* (seguridad, transacciones, confiabilidad) del que muchas organizaciones grandes todavía dependen. Cambiar el formato de un contrato ya publicado y en producción durante 20 años tiene un costo altísimo.

¿Qué pasa si mi consumidor de RabbitMQ se cae a mitad de procesar un mensaje?

Depende de si confirmaste (ack) el mensaje antes o después de caerte. Con confirmación manual, si te caes antes de confirmar, RabbitMQ reencola el mensaje para que otro consumidor (o tú al reiniciar) lo vuelva a procesar — por eso el consumidor debe ser idempotente.

¿Debo usar Kafka en vez de RabbitMQ si mi proyecto final lo requiere?

Solo si tu caso de uso realmente necesita las características de Kafka (volumen masivo, reproceso de eventos, streaming). Para la mayoría de proyectos académicos, RabbitMQ es más que suficiente y bastante más simple de operar.

¿Por qué el frontend del Reto 1 usa NestJS en modo MVC en vez de una API REST + un frontend separado?

Porque el objetivo pedagógico de esta semana es la integración de backend (SOA + ESB + mensajería), no el desarrollo de una interfaz compleja. Renderizar HTML desde el servidor permite enfocar el esfuerzo en las cuatro piezas evaluadas, sin la complejidad adicional de un frontend SPA independiente.

Autoevaluación

1. Explica la diferencia entre el contrato WSDL de SOA y el contrato OpenAPI de REST.
2. ¿Cuáles son las tres funciones centrales de un ESB? Da un ejemplo propio de cada una.
3. Compara el patrón cola y el patrón tópico con un caso de tu propio proyecto.
4. ¿Por qué Kafka permite reprocesar eventos y RabbitMQ generalmente no?
5. Describe el flujo completo del Reto 1 (las 4 piezas) sin ver el diagrama.

Glosario rápido

- WSDL: contrato XML que describe un servicio SOAP.
- ESB: componente de mediación que enruta, transforma y opcionalmente orquesta mensajes.
- Exchange (RabbitMQ): recibe mensajes del productor y los enruta a colas según reglas.
- Partición (Kafka): subdivisión de un tópico que permite paralelismo y orden dentro de sí misma.
- Competing Consumers: patrón donde varios consumidores compiten por los mensajes de una misma cola.
- Dead Letter Queue: cola donde se apartan mensajes que fallan repetidamente.
- soap (npm): paquete de Node.js para exponer/consumir servicios SOAP.
- amqplib (npm): cliente de Node.js para el protocolo AMQP 0-9-1 (RabbitMQ).
- NestJS modo MVC: configuración de NestJS con motor de plantillas (hbs) para renderizar HTML en el servidor.

Referencias

- Hohpe, G. & Woolf, B. (2003). Enterprise Integration Patterns. Addison-Wesley.
- RabbitMQ Documentation: rabbitmq.com/documentation.html
- Apache Kafka Documentation: kafka.apache.org/documentation
- soap (npm): npmjs.com/package/soap
- amqplib (npm): npmjs.com/package/amqplib
- NestJS - MVC: docs.nestjs.com/techniques/mvc
- W3C. Web Services Description Language (WSDL) 1.1.